# Listen

Learning Doc 05: The Local Data Model
What Listen remembers, where, for how long, and who may see it

Detailed design, Section 4

13 October 2026

**Abstract**

Everything Listen does produces data: commands you gave, sentences it overheard, the apps you used. This document designs the database on your laptop that holds it: the tables, the columns, which data is private and which may be synced, how private data deletes itself after 30 days, how you mark mistakes, and how the database can change shape later without losing anything. You already know tables from pandas; here you will see how a real database thinks about them.

## Contents

**1 Decisions this section is built on** **2**

**2 From DataFrames to a database** **2**

**3 The privacy split, as a rule** **2**

**4 The tables in detail** **3**
4.1 `commands` (syncable) . . . 4
4.2 `continuous_log` (private) . . . 4
4.3 `app_sessions` (private) . . . 4
4.4 `usage_daily` (syncable) . . . 4
4.5 `meta` . . . 5

**5 Marking mistakes (“that’s wrong”)** **5**

**6 Data retention: the 30-day purge** **5**

**7 No audio on disk** **6**

**8 Making the database safe to use from several threads** **6**

**9 Changing the database later: migrations** **6**

**10 Glossary** **7**

**11 Check your understanding** **7**

# 1 Decisions this section is built on

| Topic | Decision |
|---|---|
| Private data lifetime | 30 days, then deleted automatically. |
| Audio recordings | Never saved to disk. Audio lives in memory only, and only until it is transcribed. |
| Marking mistakes | Both: say "that's wrong" right away, *and* a review list for later. |
| Sync (from Doc 01) | Commands and summaries may go to the cloud; overheard speech and raw window titles never do. |

# 2 From DataFrames to a database

**Concept: Relational database**

A **relational database** stores data in **tables**: rows (one per thing, e.g. one command) and columns (one per property, e.g. the time). That is exactly a pandas DataFrame. The differences: a database lives on disk and survives restarts, many parts of a program can read and write it safely, it can enforce rules ("this column may never be empty"), and it answers questions in **SQL** without loading everything into memory.

| pandas (what you know) | SQL (what Listen uses) |
|---|---|
| `df[df.result == "executed"]` | `SELECT * FROM commands WHERE result = 'executed'` |
| `df.groupby("app").seconds.sum()` | `SELECT app, SUM(seconds) FROM usage_daily GROUP BY app` |
| `pd.concat([df, new_row])` | `INSERT INTO commands (...) VALUES (...)` |

**Concept: SQLite**

**SQLite** is a complete SQL database stored in **one file**, with no server program running. Python includes it (`import sqlite3`): nothing to install, almost no memory. Phones, browsers and most desktop apps use it for exactly this kind of local data.

The file lives at `%APPDATA%\Listen\listen.db`, outside the code repository (Doc 02: your data never goes in git). When we build, you will be able to open it with pandas: `pd.read_sql("SELECT * FROM commands", conn)`.

# 3 The privacy split, as a rule

Every table is either **syncable** (may go to the cloud) or **private** (never leaves the laptop, deleted after 30 days). A table never mixes the two, so the sync code can follow one rule: *only read syncable tables.*

| Table | Kind | Holds |
|---|---|---|
| `commands` | syncable | Everything you asked Listen to do, and what happened |
| `usage_daily` | syncable | Seconds per app and per site, per day |
| `continuous_log` | private, 30 days | What continuous mode overheard that was not a command |
| `app_sessions` | private, 30 days | Each stretch of time an app was in front, with its raw window title |
| `meta` | local only | Database version and housekeeping dates |

**Watch out**

**A subtle privacy rule.** In wake-word and push-to-talk modes, a sentence that does *not* match a command was still *spoken to Listen*, so it goes into `commands` as `not_matched` (useful: "commands you tried that failed"). In continuous mode, an unmatched sentence is just someone in the room talking, so it goes to the private `continuous_log` instead. Same kind of data, different table, because the *situation* decides whether it is private.

## 4 The tables in detail

**Concept: Primary key, UUID and UTC timestamps**

Every table has an `id`: a number the database assigns, unique within the table (the **primary key**). Syncable tables also get a `uuid`: a random 128-bit identifier that is unique *everywhere*, so the server can recognise "I already have this row" even if it is sent twice. All times are stored in **UTC** (world time, no daylight saving) and converted to Indian time only for display; mixing time zones in stored data is a classic source of bugs.

### 4.1 `commands` (syncable)

| Column | Type | Meaning |
|---|---|---|
| `id`, `uuid`, `user_id` | int, text, text | Identity (Doc 01: `user_id` from day one) |
| `ts` | text (UTC) | When you finished speaking |
| `mode` | text | `wake`, `push_to_talk` or `continuous` |
| `heard` | text | The transcribed sentence |
| `command_id` | text | Which command matched (empty if none) |
| `slots` | text (JSON) | Filled slots, e.g. `{"query": "what is an atom"}` |
| `action` | text | The primitive (or `routine`) |
| `result` | text | `executed`, `failed`, `not_matched`, `low_confidence`, `cancelled` |
| `error` | text | What went wrong, if anything |
| `stt_conf`, `match_conf` | real | The two confidence scores (Doc 03 §11) |
| `ms_silence`, `ms_stt`, `ms_action`, `ms_total` | int | The latency budget, measured (Doc 04 §8) |
| `model` | text | Which speech model produced `heard` |
| `undone` | int (0/1) | Whether you undid it |
| `verdict` | text | Empty, `right` or `wrong` (Section 5) |
| `verdict_source` | text | `voice` or `review` |
| `synced_at` | text | Empty until uploaded (used by sync, Section 5 of the design) |

**Why it is done this way**

Timings, confidences and verdicts live on the *same* row as the command. That one table is then enough for calibration ("what confidence did the wrong guesses have?"), for the latency report, and for the "commands that failed" list, each a single SQL query or one `pd.read_sql`.

### 4.2 `continuous_log` (private)

`id`, `ts`, `heard`, `stt_conf`. Nothing else: the smallest record that still lets you read back what was overheard.

### 4.3 `app_sessions` (private)

`id`, `started`, `ended`, `process` (e.g. `chrome.exe`), `app` ("Google Chrome"), `raw_title` (private), `site` (cleaned, e.g. "Google Search"; designed with the usage tracker). One row per stretch of time an app stayed in front.

### 4.4 `usage_daily` (syncable)

`date`, `app`, `site`, `seconds`, plus `uuid`, `user_id`, `synced_at`. One row per app/site per day.

**Concept: Aggregation (roll-up)**

**Aggregation** turns many detailed rows into a few summary rows: 600 sessions in a day become "Chrome: 3h 12m, of which Google Search: 40m". A **roll-up** job does this every hour from `app_sessions` into `usage_daily`. Because the summary is computed *before* the 30-day purge, deleting old private sessions never deletes your long-term statistics. It is the same as a pandas `groupby` whose result you keep after dropping the raw data.

### 4.5 `meta`

A two-column key/value table: `schema_version`, `last_purge`, `last_rollup`.

## 5 Marking mistakes ("that's wrong")

**By voice.** "That's wrong" (also "wrong", "that was wrong") is a built-in command. It sets `verdict = wrong`, `verdict_source = voice` on the most recent command row, plays the "done" chime and shows "Marked as wrong". It does *not* undo anything; "undo" remains a separate command, so you decide which you want.

**By review.** A developer command lists recent commands with their scores:

```
python -m listen_agent review --last 20      # [r]ight / [w]rong / [d]elete
  #12  09:41  "search what is an atom"   search   stt 0.93  match 1.00   [r]ight / [w]
    rong / skip
  #13  09:44  "close crow"               close    stt 0.61  match 0.82   [r]ight / [w]
    rong / skip
```

The web app gets the same list later. Rows you never mark stay empty, and calibration only uses marked rows.

**Why it is done this way**

Calibration needs *labelled* examples, just like training a classifier. Voice marking catches mistakes in the moment; review catches the ones you only notice later, and also lets you mark *right* answers, which calibration needs just as much as wrong ones.

## 6 Data retention: the 30-day purge

**Concept: Retention policy**

A **retention policy** states how long each kind of data is kept. Keeping data "forever, just in case" is a privacy risk and a liability; deleting it on a clear schedule is what careful systems do (and what data-protection laws often require).

The purge runs at start-up and then every 6 hours:

```
1. run the roll-up first (so usage_daily is up to date)
2. DELETE FROM continuous_log WHERE ts      < now - 30 days
3. DELETE FROM app_sessions   WHERE ended   < now - 30 days
4. record last_purge in meta
```

`commands` and `usage_daily` are not purged. They are small (a few hundred bytes per command; a year of heavy use is a few megabytes) and they are the history your analytics are built on.

**Deleting on request (decided).** Command history is kept **until you choose to delete it**. You can delete single commands from the review list (`[d]elete`) and later from the web app. A deletion must also reach the cloud copy; how that works ("tombstones") is part of the sync design, Section 5.

## 7 No audio on disk

Audio exists only as an array in memory: from the microphone, through the listener, into the transcriber, then it is discarded. No table has an audio column, and no code path writes a sound file.

**Watch out**

This affects the speed benchmark from Doc 04, which planned to "record about 20 commands as audio files". The benchmark is redesigned to respect this rule: in **benchmark mode** you speak each test command live; the clip is held in memory, transcribed by `tiny.en` *and* `base.en` one after the other (so both models get identical audio, a fair comparison), the two results and timings are stored as text, and the clip is discarded. The cost: you speak the test set once per benchmark run instead of reusing recordings.

## 8 Making the database safe to use from several threads

**Concept: Write-ahead logging (WAL)**

Normally SQLite blocks readers while someone writes. In **WAL mode**, new writes go to a side file first, so the review command (or a pandas notebook) can read the database *while* the agent keeps writing. One line turns it on: `PRAGMA journal_mode=WAL`.

Together with Doc 04's rule that *only the storage thread writes*, this gives safe access: one writer, any number of readers. The storage thread writes in small **transactions** (a group of changes that is saved completely or not at all), batching events every second, so a crash can never leave half a command written.

**Concept: Index**

An **index** is a sorted lookup structure on a column, like the index at the back of a book: instead of reading every row to find commands from today, the database jumps straight there. We add indexes on `commands.ts` and `synced_at`, and on the time columns the purge uses. Each index makes writes slightly slower, so we add only those we query.

## 9 Changing the database later: migrations

**Concept: Schema migration**

Your tables will change: next month you may want a `routine_step` column. You cannot just edit the `CREATE TABLE` line, because your real database already exists with real data. A **migration** is a small numbered script that changes an existing database from one version to the next ("add column X"). On start-up the agent reads `schema_version` from `meta` and runs every migration newer than it, in order.

```
agent/listen_agent/storage/migrations/
    001_initial.sql        -- creates the five tables
```

```
002_add_routine_step.sql   (example of a future change)
```

**Why it is done this way**

Without migrations, every change to the tables would mean deleting your database and losing your history. With them, your laptop's database upgrades itself safely, and the server's database (Section 6 of the design) will use the same idea.

## 10 Glossary

**Aggregation / roll-up**
Turning detailed rows into summary rows, kept separately.

**Index**
A lookup structure that makes searching a column fast.

**Migration**
A numbered script that upgrades an existing database's structure.

**Primary key**
A column that uniquely identifies each row in a table.

**Relational database**
Data stored as tables of rows and columns, queried with SQL.

**Retention policy**
How long each kind of data is kept before deletion.

**SQL** The language for querying and changing relational databases.

**SQLite**
A full SQL database in a single file, built into Python.

**Transaction**
A group of changes saved completely or not at all.

**UTC** World time with no time zones or daylight saving; how all times are stored.

**UUID**
A random identifier unique across all machines.

**WAL**
SQLite mode letting readers read while one writer writes.

## 11 Check your understanding

**Check yourself**

1. In continuous mode, someone says "let's get food". Which table does it go to, and why not `commands`?
2. The purge deletes `app_sessions` older than 30 days. Why does your "Chrome usage in March" chart still work in June?
3. Why is every time stored in UTC rather than Indian time?
4. Write the SQL for: average `ms_total` of executed commands today.
5. You add a column next month. Why can't you just edit the `CREATE TABLE` line?

6. Why does the review list let you mark *right* answers too?
7. How does the benchmark compare two models fairly without saving audio?